CORE–2: DIGITAL LOGIC – Unit-1

Character Codes-Representation of alphanumeric characters in bits 0 and 1 is done by character codes. There are three widely used character codes:

- Binary Coded Decimal(BCD)
- Excess-3 Code
- American standard Code for Information Interchange (ASCII)
- Extended Binary Coded Decimal Interchange Code(EBCDIC)

Binary Coded Decimal (BCD): BCD uses 6 bits and can represent $2^6 = 64$ characters.

Excess-3 Code: The excess-3 code is also known as **XS-3 code**. We can easily get an excess-3 code of a decimal number by simply adding 3 to each decimal digit. And then we write the 4-bit binary number for each digit of the decimal number. We can also add 0011 in each 4-bit BCD code of the decimal number for getting excess-3 code.

Decimal Digit	BCD Code	Excess-3 Code
0	0000	0011
1	0001	0100
2	0010	0101
⋮	⋮	⋮
9	1001	1100

American standard Code for Information Interchange (ASCII):

ASCII-7: It uses 7 bits and can represent 2^7 =128 characters. These characters represent 26 upper case letters (A to Z), 26 lowercase letters (a to z), 10 numbers (0 to 9), 33 special characters and symbols and 33 control characters.

ASCII-8: It uses 8 bits and can represent 2^8 =256 characters. For all practical purposes an 8-bit code is used.

NUMBER SYSTEMS: Many number systems are in use in digital technology. The most common are the decimal, binary, octal, and hexadecimal systems. The decimal system is clearly the most familiar to us because it is a tool that we use every day. Examining some of its characteristics will help us to better understand the other systems.

Decimal System: The decimal system is composed of 10 numerals or symbols. These symbols are 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9. Using these symbols as digits of a number, we can express any quantity. The decimal system is also called the base-10 system because it has 10 digits for example 3.1410, 5210 etc.

Binary System: In the binary system, there are only two symbols or possible digit values, 0 and 1. This base-2 system can be used to represent any quantity that can be represented in binary system. In digital systems the information that is being processed is usually presented in binary form. Binary quantities can be represented by any device that has only two operating states or possible conditions. E.g. a switch is only open or closed. We arbitrarily (as we define them) let an open switch represent binary 0 and a closed switch represent binary 1.

Octal System: The octal number system has a base of eight, meaning that it has eight possible digits: 0,1,2,3,4,5,6 and7.

Hexadecimal System: The hexadecimal system uses base 16. Thus, it has 16 possible digit symbols. It uses the digits 0 through 9 and the letters A, B, C, D, E, and F as the 16 digit symbols.

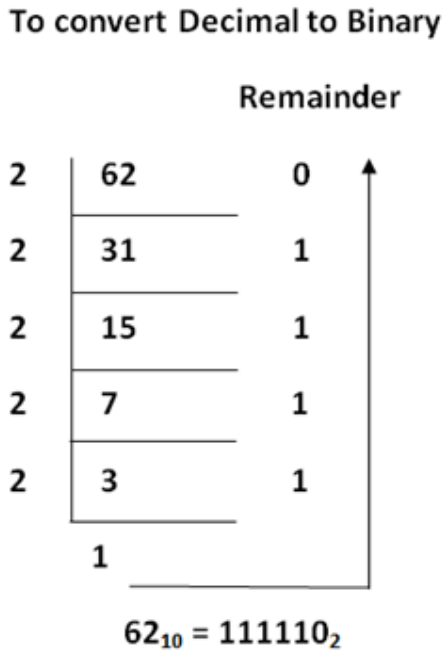
Code Conversion: Converting from one code form to another code form is called code conversion, like converting from binary to decimal or converting from hexadecimal to decimal etc.

Binary-to-Decimal Conversion: Any binary number can be converted to its decimal equivalent simply by summing together the weights of the various positions in the binary number which contain a 1.
Example: $(11011)_2$ = (_____)$_{10}$
Solution: $1*2^4 + 1*2^3 + 0*2^2 + 1*2^1 + 1*2^0 = 16+8+0+2+1 = (27)_{10}$

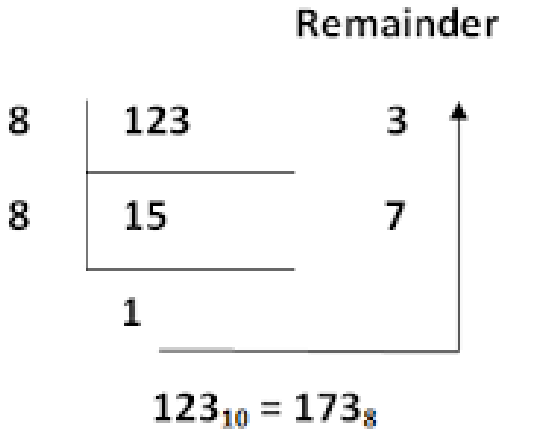
Octal-to-Decimal Conversion: Example: $(274)_8$ = (_____)$_{10}$
Solution: $2*8^2 + 7*8^1 + 4*8^0 = 128+56+4 = (188)_{10}$

Hexadecimal-to-Decimal Conversion: Example: $(2A)_{16}$ = (_____)$_{10}$
Solution: $2*16^1 + 10*16^0 = 32+10 = (42)_{10}$

Decimal-to- Binary Conversion



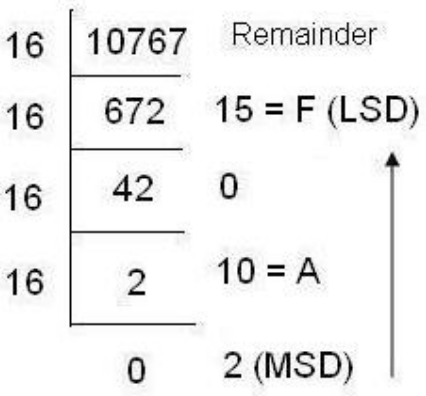
Decimal-to- Octal Conversion



Decimal-To- Hexadecimal Conversion:

Example: $(10767)_{10}$ = (_____)$_{16}$

Solution: $(10767)_{10}$ = (2A0F)$_{16}$



Octal-To-Binary Conversion: Each Octal digit is represented by three binary digits.
Example: $(472)_8 = (100\ 111\ 010)_2$

Octal-To-Hexadecimal and Hexadecimal-To-Octal Conversion:

- Convert Octal (Hexadecimal) to Binary first.
- Regroup the binary number by three bits per group starting from LSB if Octal is required.
- Regroup the binary number by four bits per group starting from LSB if Hexadecimal is required.

Example-1: $(472)_8 = (100\ 111\ 010)_2 = (1\ 0011\ 1010)_2 = (13A)_{16}$
Example-2: $(4B5)_{16} = (100\ 1011\ 0101)_2 = (10\ 010\ 110\ 101)_2 = (2265)_8$

BOOLEAN ALGEBRA: Boolean algebra is used to analyze and simplify the digital (logic) circuits. It uses only the binary numbers i.e. 0 and 1. It is also called as Binary Algebra or logical Algebra. Boolean algebra was invented by George Boole in 1854.

Rule in Boolean algebra: Following are the important rules used in Boolean algebra.

- Variable used can have only two values. Binary 1 for HIGH and Binary 0 for LOW.
- Complement of a variable is represented by an over bar (-). Thus, complement of variable B is represented as $\overline{B}$. Thus if B = 0 then $\overline{B}$ = 1 and B = 1 then $\overline{B}$ = 0.
- ORing of the variables is represented by a plus (+) sign between them. For example ORing of A, B, C is represented as A + B + C.
- Logical ANDing of the two or more variable is represented by writing a dot between them such as A.B.C. Sometime the dot may be omitted like ABC.

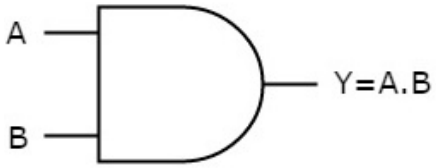
Boolean Laws: There are seven of Boolean Laws.		
Sl No.	Name of the law	Law
1	Commutative law	i. A.B = B.A ii. A + B = B + A
2	Associative law	i. (A.B).C = A.(B.C) ii. (A+B)+C = A+(B+C)
3	Distributive law	i. A.(B+C) = A.B + A.C ii. A+BC = (A+B).(A+C)
4	DeMorgan's law	$\overline{x \cdot y} = \overline{x} + \overline{y}$ $\overline{x + y} = \overline{x} \cdot \overline{y}$

5	AND law	(i) $A.0 = 0$ (ii) $A.1 = A$ (iii) $A.A = A$ (iv) $A.\overline{A} = 0$
6	OR law	(i) $A + 0 = A$ (ii) $A + 1 = 1$ (iii) $A + A = A$ (iv) $A + \overline{A} = 1$
7	Inversion law	$\overline{\overline{A}} = A$

LOGIC GATES: Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on certain logic. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc.

AND Gate:
A circuit which performs an AND operation is shown in figure. It has n input (n >= 2) and one output.

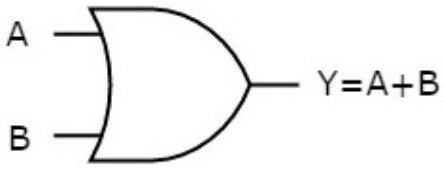
Logic diagram



Truth Table		
Inputs		Output
A	B	Y = AB
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate:
A circuit which performs an OR operation is shown in figure. It has n input (n >= 2) and one output.

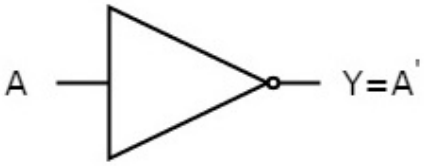
Logic diagram



Truth Table		
Inputs		Output
A	B	Y = A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate:
NOT gate is also known as Inverter. It has one input A and one output Y.

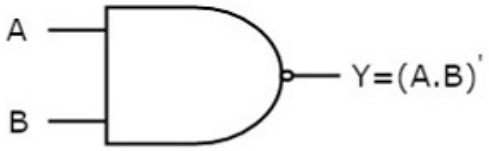
Logic diagram



Truth Table	
Input	Output
A	Y = A'
0	1
1	0

NAND Gate:
A NOT-AND operation is known as NAND operation. It has n input (n >= 2) and one output.

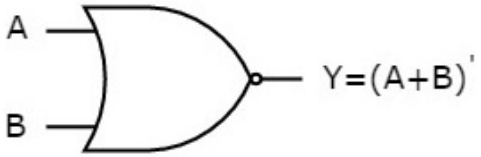
Logic diagram



Truth Table		
Inputs		Output
A	B	Y = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate:
A NOT-OR operation is known as NOR operation. It has n input (n >= 2) and one output.

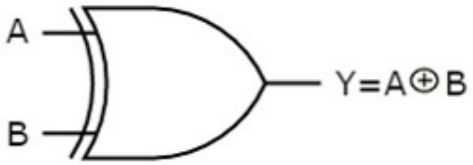
Logic diagram



Truth Table		
Inputs		Output
A	B	Y = (A+B)'
0	0	1
0	1	0
1	0	0
1	1	0

X-OR Gate:
X-OR or EX-OR gate is a special type of gate. It can be used in the half adder, full adder and subtractor.

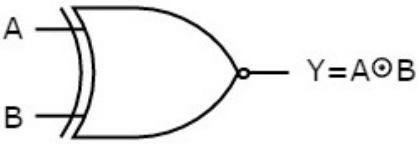
Logic diagram



Truth Table		
Inputs		Output
A	B	Y = A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

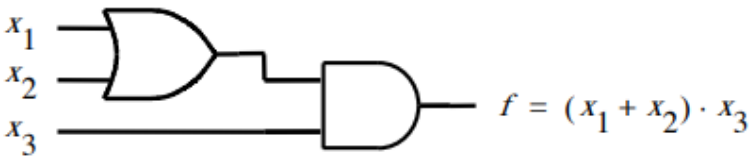
X-NOR Gate:
X-NOR gate is a special type of gate. It can be used in the half adder, full adder and subtractor.

Logic diagram



Truth Table		
Inputs		Output
A	B	**Y = A⊙B**
0	0	1
0	1	0
1	0	0
1	1	1

<u>Basic Logic Functions</u>:
The basic logic functions are AND, OR and NOT. The logic diagram shown here implements the logic function $f = (x_1 + x_2).x_3$



<u>Canonical & Standard Forms</u>: We will get four Boolean product terms by combining two variables x and y with logical AND operation. These Boolean product terms are called as **min terms** or **standard product terms**. The min terms are x'y', x'y, xy' and xy.

Similarly, we will get four Boolean sum terms by combining two variables x and y with logical OR operation. These Boolean sum terms are called as **Max terms** or **standard sum terms**. The Max terms are x + y, x + y', x' + y and x' + y'.

The table shown here represent the MIN terms and MAX terms for 2 variables.

If the binary variable is '0', then it is represented as complement of variable in min term and as the variable itself in Max term. Similarly, if the binary variable is '1', then it is represented as complement of variable in Max term and as the variable itself in min term.

x	y	Min terms	Max terms
0	0	m_0=x'y'	M_0=x + y
0	1	m_1=x'y	M_1=x + y'
1	0	m_2=xy'	M_2=x' + y
1	1	m_3=xy	M_3=x' + y'

From the above table, we can easily notice that min terms and Max terms are complement of each other. If there are 'n' Boolean variables, then there will be 2^n min terms and 2^n Max terms.

Canonical SoP and PoS forms: A Boolean function can be expressed by the following two ways.

- Canonical SoP (Sum of Products) form
- Canonical PoS (Product of Sums) form

Canonical SoP form: In this form, each product term contains all literals. These product terms are nothing but the min terms. Hence, canonical SoP form is also called as **sum of Min terms** form.

First, identify the min terms for which, the output variable is one and then do the logical OR of those min terms in order to get the Boolean expression function corresponding to that output variable. This Boolean function will be in the form of sum of min terms.

Follow the same procedure for other output variables also, if there is more than one output variable.

Example: Consider the **truth table** shown here.

Inputs			Output
p	q	r	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Here, the output f is '1' for four combinations of inputs. The corresponding min terms are p'qr, pq'r, pqr', pqr. By doing logical OR of these four min terms, we will get the Boolean function of output f.

Therefore, the Boolean function of output is, f = p'qr + pq'r + pqr' + pqr. This is the **canonical SoP form** of output f. We can also represent this function in following two notations.

$$f = m_3+m_5+m_6+m_7 \quad \text{or} \quad f = \sum m(3,5,6,7)$$

In one equation, we represented the function as sum of respective min terms. In other equation, we used the symbol for summation of those min terms.

Canonical PoS form: In this form, each sum term contains all literals. These sum terms are nothing but the Max terms. Hence, canonical PoS form is also called as **product of Max terms** form.

First, identify the Max terms for which, the output variable is zero and then do the logical AND of those Max terms in order to get the Boolean expression function corresponding to that output variable. This Boolean function will be in the form of product of Max terms. Follow the same procedure for other output variables also, if there is more than one output variable.

Example: Consider the same truth table of previous example. Here, the output f is '0' for four combinations of inputs. The corresponding Max terms are p+q+r, p+q+r', p+q'+r, p'+q+r. By doing logical AND of these four Max terms, we will get the Boolean function of output f.

Therefore, the Boolean function of output is, f = (p+q+r)(p+q+r′)(p+q′+r)(p′+q+r). This is the **canonical PoS form** of output f. We can also represent this function in following two notations.

$f = M_0.M_1.M_2.M_4$ or $f = \prod M(0,1,2,4)$

In one equation, we represented the function as product of respective Max terms. In other equation, we used the symbol for multiplication of those Max terms.

Inputs			Output
p	**q**	**r**	**f**
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

The Boolean function, f = (p′+q+r)(p+q′+r)(p+q+r′)(p+q+r) is the dual of the Boolean function, f = p'qr + pq'r + pqr' + pqr. Therefore, both canonical SoP and canonical PoS forms are **Dual** to each other. Functionally, these two forms are same. Based on the requirement, we can use one of these two forms.

Principle of Duality: It states that the **dual** of the Boolean function is obtained by interchanging the logical AND operator with logical OR operator and zeros with ones. For every Boolean function, there will be a corresponding Dual function. E.g x+0 = x dual of this is x.1 = x

Standard SoP and PoS forms: We discussed two canonical forms of representing the Boolean outputs. Similarly, there are two standard forms of representing the Boolean outputs. These are the simplified version of canonical forms.

- Standard SoP(Sum of Products) form
- Standard PoS(Product of Sums) form

The main **advantage** of standard forms is that the number of inputs applied to logic gates can be minimized. Sometimes, there will be reduction in the total number of logic gates required.

Standard SoP form: In this form, each product term need not contain all literals. So, the product terms may or may not be the min terms. Therefore, the Standard SoP form is the simplified form of canonical SoP form. We will get Standard SoP form of output variable in two steps.

- Get the canonical SoP form of output variable

- Simplify the above Boolean function, which is in canonical SoP form.

Follow the same procedure for other output variables also, if there is more than one output variable. Sometimes, it may not possible to simplify the canonical SoP form. In that case, both canonical and standard SoP forms are same.

Example: Convert the Boolean function f = p'qr + pq'r + pqr' + pqr which is in Canonical SoP form. Represent this function in Standard SoP form.

Solution: Given f = p'qr + pq'r + pqr' + pqr

⇒ f = p'qr + pq'r + pqr' + pqr + pqr+ pqr (∵ x + x = x)

⇒ f = (p'qr + pqr) + (pq'r + pqr) + (pqr' + pqr)

⇒ f = (p' + p)qr + (q' + q)pr + pq(r' + r)

⇒ f = qr + pr + pq

This is the simplified Boolean function. Therefore, the **standard SoP form** corresponding to given canonical SoP form is **f = pq + qr + pr**

Standard PoS form: In this form, each sum term need not contain all literals. So, the sum terms may or may not be the Max terms. Therefore, the Standard PoS form is the simplified form of canonical PoS form.

We will get Standard PoS form of output variable in two steps.

- Get the canonical PoS form of output variable
- Simplify the above Boolean function, which is in canonical PoS form.

Follow the same procedure for other output variables also, if there is more than one output variable. Sometimes, it may not possible to simplify the canonical PoS form. In that case, both canonical and standard PoS forms are same.

Example: Convert the Boolean function f = (p+q+r)(p+q+r′)(p+q′+r)(p′+q+r) which is in Canonical PoS form. Represent this function in Standard PoS form.

Solution: Given f = (p+q+r)(p+q+r′)(p+q′+r)(p′+q+r)

⇒ f = (p+q+r)(p+q+r)(p+q+r)(p+q+r′)(p+q′+r)(p′+q+r) (∵ x.x = x)

⇒ f = [(p+q+r)(p+q+r′)] [(p+q+r) (p+q′+r)] [(p+q+r) (p′+q+r)]

⇒ f = [(p+q)+rr′)] [(p+r)+ qq′] [(q+r)+pp′] [∵ x+y.z = (x+y)(x+z)]

⇒ f = (p+q)(p+r)(q+r) [∵ xx′ = 0 and x+0 = x]

This is the simplified Boolean function. Therefore, the **standard PoS form** corresponding to given canonical PoS form is **f = (p+q)(q+r)(p+r).** This is the **dual** of the Boolean function, f = pq + qr + pr. Therefore, both Standard SoP and Standard PoS forms are Dual to each other.

Minimization of Logic Expressions: Minimization of logic expression is required to minimize the number of logic gates in a logic circuit which further minimizes the space, power consumption and cost of the circuit. The minimization can be achieved by the following ways:

1. By algebraic method
2. By Karnaugh map (K-Map)

Minimization by algebraic method: This can be achieved by using Boolean postulates and theorems.

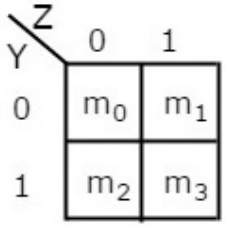
Example: Minimize the Boolean function f = x'y + xy' + xy

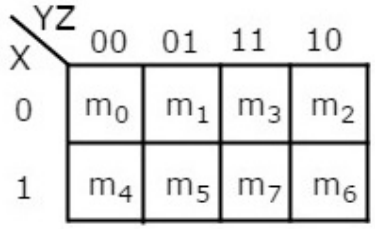
Solution: x'y + xy' + xy = x'y + xy + xy'
= (x' + x)y + xy'
= y + xy'
= (y + x) (y + y')
= x + y

Minimization by Karnaugh map (K-Map): In algebraic method, we simplify a Boolean functions using Boolean postulates and theorems. It is a time consuming process and we have to re-write the simplified expressions after each step.

To overcome this difficulty, **Karnaugh** introduced a method for simplification of Boolean functions in an easy way. This method is known as **Karnaugh map method or K-map method.** It is a graphical method, which consists of 2^n cells for 'n' variables. The adjacent cells are differed only in single bit position.

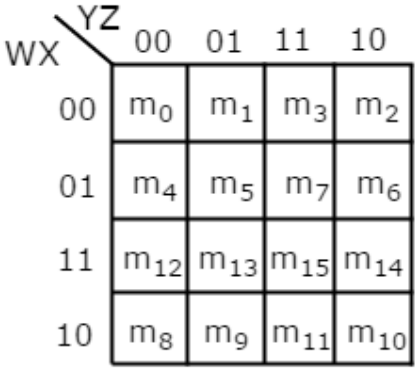
2 Variable K-Map: The number of cells in 2 variable K-map is four, since the number of variables is two. The following figure shows 2 variable K-Map.





3 Variable K-Map: The number of cells in 3 variable K-map is eight, since the number of variables is three. The following figure shows 3 variable K-Map.

4 Variable K-Map: The number of cells in 4 variable K-map is sixteen, since the number of variables is four. The following figure shows 4 variable K-Map.



Minimization of Boolean Functions using K-Maps: If we consider the combination of inputs for which the Boolean function is **'1'**, then we will get the Boolean function, which is in **standard sum of products** form after simplifying the K-map.

Similarly, if we consider the combination of inputs for which the Boolean function is **'0'**, then we will get the Boolean function, which is in **standard product of sums** form after simplifying the K-map.

Rules for simplifying K-maps in order to get standard sum of products form:

- Select the respective K-map based on the number of variables present in the Boolean function.
- If the Boolean function is given as sum of min terms form, then place the ones at respective min term cells in the K-map.
- Check for the possibilities of grouping maximum number of adjacent ones. It should be **powers of two**. Start from highest power of two and upto least power of two. Highest power is equal to the number of variables considered in K-map and least power is zero.

Note 1 − If outputs are not defined for some combination of inputs, then those output values will be represented with **don't care symbol 'x'**.

Note 2 − If don't care terms are also present, then place 'x' in the respective don't cares cells of K-map. Consider only the don't cares 'x' that are helpful for grouping maximum number of adjacent ones. In those cases, the don't care value is treated as '1'.

Example: Let us simplify the following Boolean function, f (W,X,Y,Z) = WX'Y' + WY + W'YZ' using K-map.

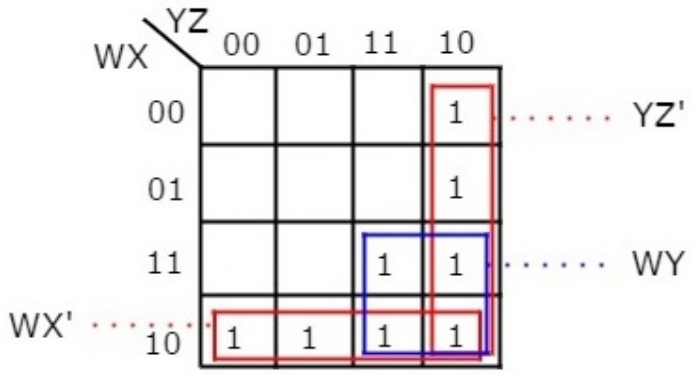
Solution: The given Boolean function is in sum of products form. It is having 4 variables W, X, Y & Z. So, we require 4-variable K-map. The 4 variable K-map with ones corresponding to the given product terms is shown in the fig.

WX \ YZ	00	01	11	10
00				1
01				1
11			1	1
10	1	1	1	1

Here, 1s are placed in the following cells of K-map.

- The cells, which are common to the intersection of Row 4 and columns 1 & 2 are corresponding to the product term, WX'Y'.
- The cells, which are common to the intersection of Rows 3 & 4 and columns 3 & 4 are corresponding to the product term, WY.
- The cells, which are common to the intersection of Rows 1 & 2 and column 4 are corresponding to the product term, W'YZ'.

There are no possibilities of grouping either 16 adjacent ones or 8 adjacent ones. There are three possibilities of grouping 4 adjacent ones. After these three groupings, there is no single one left as ungrouped. So, we no need to check for grouping of 2 adjacent ones. The 4 variable K-map with these three groupings is shown in the fig.



The **simplified Boolean function** is **f = WX' + WY + YZ'**

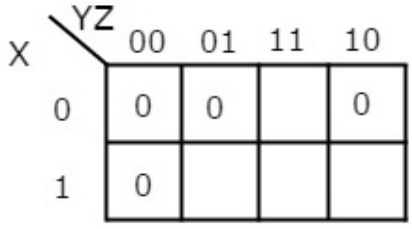
Rules for simplifying K-maps in order to get standard product of sums form:

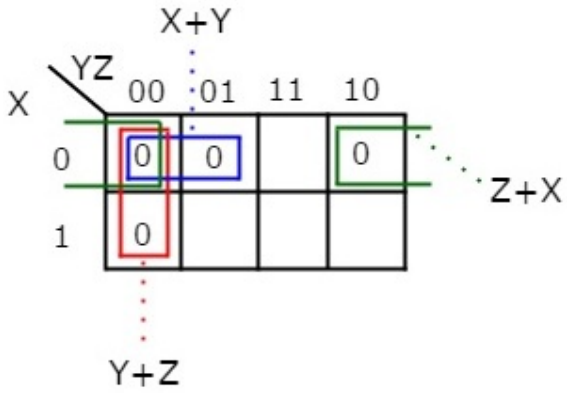
- Select the respective K-map based on the number of variables present in the Boolean function.
- If the Boolean function is given as product of Max terms form, then place the zeroes at respective Max term cells in the K-map.
- Check for the possibilities of grouping maximum number of adjacent zeroes. It should be powers of two. Start from highest power of two and up to least power of two. Highest power is equal to the number of variables considered in K-map and least power is zero.

Note − If don't care terms are also present, then place don't cares 'x' in the respective cells of K-map. Consider only the don't cares 'x' that are helpful for grouping maximum number of adjacent zeroes. In those cases, the don't care value is treated as '0'.

Example: Simplify the Boolean function, f(X,Y,Z) = ∏M(0,1,2,4) using K-map.

Solution: The given Boolean function is in product of Max terms form. It is having 3 variables X, Y & Z. So, we require 3 variable K-map. The given Max terms are M_0, M_1, M_2 & M_4. The 3 **variable K-map** with zeroes corresponding to the given Max terms is shown in the following figure.





There are no possibilities of grouping either 8 adjacent zeroes or 4 adjacent zeroes. There are three possibilities of grouping 2 adjacent zeroes. After these three groupings, there is no single zero left as ungrouped. The **3 variable K-map** with these three **groupings** is shown in the fig.

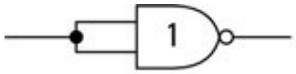
Here, we got three prime implicants X + Y, Y + Z & Z + X.

The **simplified Boolean function** is **f = (**X+Y)**(**Y+Z)(Z+X)

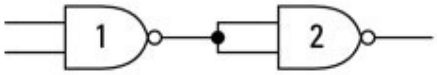
Synthesis with NAND and NOR Gates: NAND and NOR gates are universal logic gates, meaning that any other logic gate be made from NAND or NOR gates.

Making any gate using only NAND gates:

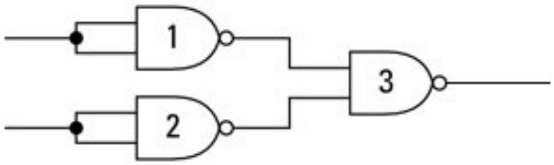
NOT



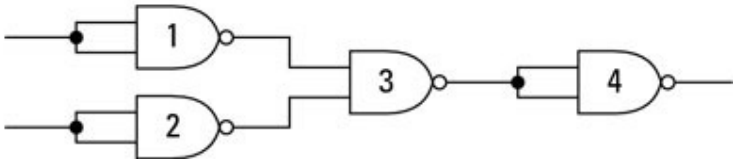
AND



OR

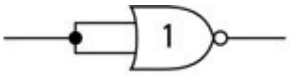


NOR

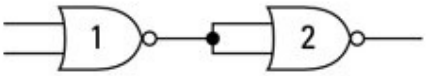


Making any gate using only NOR gate:

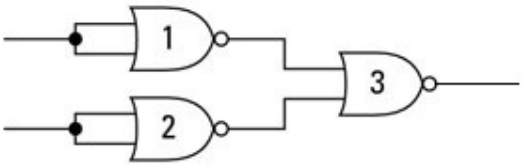
NOT



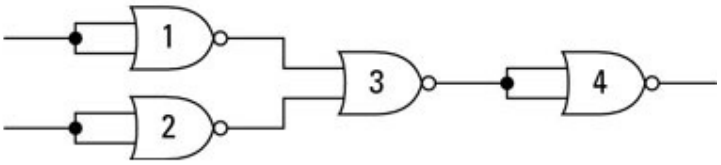
OR



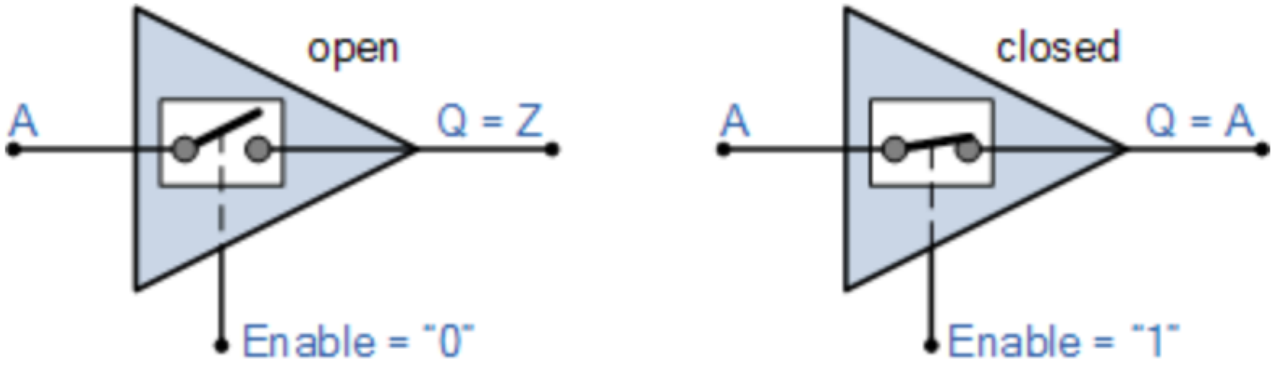
AND



NAND



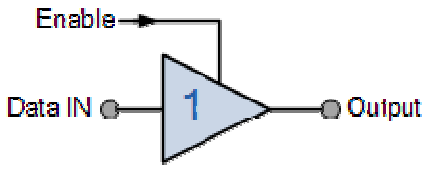
Tri-State Buffers: A Tri-state Buffer can be thought of as an input controlled switch with an output that can be electronically turned "ON" or "OFF" by means of an external "Control" or "**Enable" (EN)** signal input. This control signal can be either a logic "0" or a logic "1" type signal. So a tri-state buffer requires two inputs, one being the data input and the other being the enable or control input as shown.



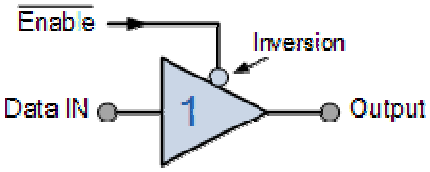
When activated into its third state it disables or turns "OFF" its output producing an open circuit condition that is neither at a logic "HIGH" or "LOW", but instead gives an output state of very high impedance, **High-Z**, or more commonly Hi-Z. Then this type of device has two logic state inputs, "0" or a "1" but can produce three different output states, "0", "1" or " Hi-Z " which is why it is called a "Tri" or "3-state" device.

There are four different types of Tri-state Buffer, one set whose output is enabled or disabled by an "**Active-HIGH**" control signal producing an inverted or non-inverted output, and another set whose buffer output is controlled by an "**Active-LOW**" control signal producing an inverted or non-inverted output as shown below.

Active "HIGH" Tri-state Buffer

Symbol	Truth Table		
	EN	IN	OUT
	0	0	Hi-Z
	0	1	Hi-Z
	1	0	0
Tri-state Buffer	1	1	1
Output = Input if Enable is equal to "1"			

Active "LOW" Tri-state Buffer

Symbol	Truth Table		
	EN	IN	OUT
	0	0	0
	0	1	1
	1	0	Hi-Z
Tri-state Buffer	1	1	Hi-Z
Output = Input if Enable is equal to "0"			